

TypeScript Syntax

Robin Langer

1 September 2026

***Companion.** This is the TypeScript reference for the note Stalin: The Game. It is not meant to be read straight through. Refer back to it as necessary.*

Contents

1	TypeScript syntax	2
1.1	Basic types	2
1.2	The weird stuff	2
1.3	Type guards	3
1.4	Subtypes	4
1.5	The powerful stuff	4
1.6	On the level of values	4
2	Structural typing	5
2.1	A tagged union writes down a finite choice	5
3	Compiling down to JavaScript	6
3.1	A cast is a promise, not a check	6
3.2	<code>unknown</code> is what you want at a boundary	6
3.3	Validate, do not cast	7

1 TypeScript syntax

It is an unfortunate accident of history that JavaScript became the “default” language for web app development. TypeScript is an unfortunate attempt to “bolt on” a type system to JavaScript after the event.

This is a “crash course” in TypeScript, and I am also learning as I write here. It is a bit of a head twister, and it bears almost no resemblance to Haskell or Lean.

Don’t try to memorize all this now. Refer back to it as necessary.

The system is deliberately unsound. This is the sentence to carry into the rest of the note. TypeScript does not claim that a well-typed program cannot go wrong. It claims to catch a useful class of mistakes.

1.1 Basic types

- The atomic types are `number`, `string`, `boolean`, `null` and `undefined`.
- A string in quotes may be used as a type. The type `"failure"` has exactly one inhabitant, namely that one string.
- The type `never` has no inhabitants at all.
- The type `A | B` is a value that is an `A` or a `B`. It is an untagged union.
- The type `{ id: number; title: string; author: string }` is a *record*. A value of it is a dictionary in the Python sense, with those three keys. In our running example this is the type of a book, and the note gives it the name `Book`.
- The type `Array<Book>` is a list whose elements are all books.
- The signature `<T, R>(x: T) => R` is a generic function.
- The keyword `export` in front of a definition means that other files may use it. Without it, the definition is private to its own file.

1.2 The weird stuff

- The type `unknown` is a value about which nothing is known. Nothing may be done to it until you have established what it is.
- The type `Record<string, unknown>` is a dictionary again, but this time with arbitrary keys and nothing at all known about the values.
- The keyword `any` switches the type checker off for that value entirely.
- The notation `u as T` says “trust me, this is a `T`”. This is very upsetting for somebody in formal verification. It is called a *type assertion*.

1.3 Type guards

The notation `u as T` above asserts a type and checks nothing. There is a second notation that checks, and it is the one the wire code in this note is built on.

```
export const isRecord = (u: unknown): u is Record<string, unknown> =>
  typeof u === "object" && u !== null && !Array.isArray(u);
```

The word `is` is a keyword. It may appear only in the return type of a function, written `u is T` where `u` is one of that function's own parameters. It says that the function returns a boolean, and that a true answer means `u` is a `T`. A function declared that way is called a *type guard*.

The type guard above is used in one place only, immediately after a `JSON.parse`. So the value it is asked about is never an arbitrary TypeScript value. It is one of the six things the JSON grammar of the previous section allows, and the test only has to tell those six apart.

The body of `isRecord` is three tests joined by `&&`. The operator `typeof` answers with a string naming the kind of a value.

The first test, `typeof u === "object"`, rules out numbers, strings and booleans. The second, `u !== null`, rules out `null`. The third, `!Array.isArray(u)`, rules out arrays, which `typeof` also calls objects. This leaves `Record<string, unknown>` as the only remaining option.

1.4 Subtypes

- In TypeScript we can talk about one type being a *subtype* of another. For example the type `"failure"` is a subtype of `string`. Every type is a subtype of `unknown`, and `never` is a subtype of every other type.
- The type `Extract<Q, S>` keeps only those members of the union `Q` that are subtypes of `S`, and throws the rest away.
- The notation `Book[K]` tells you the type of the value stored at field `K` of the record type `Book`. It is called an *indexed access type*. So `Book["id"]` is `number`, and `Book["title"]` is `string`.
- The constraint `K extends U` says that `K` must be a subtype of `U`.

1.5 The powerful stuff

- The notation `X extends Y ? A : B` is a *conditional type*. It computes a type by testing `X` against `Y`. This is the construct everything else in this note rests on, and it gets a subsection of its own below.
- The notation `{ [K in U]: F<K> }` is a *mapped type*, with one field for every member of the union `U`. This one also gets a subsection of its own below.

1.6 On the level of values

- The keywords `interface` and `type` both name a type, the first restricted to records.
- The declaration `const x: T = e` introduces a constant called `x`, gives it the type `T`, and sets it equal to the value `e`. The type may be left out. Writing `const n = 3` gives `n` the type `number`, because `3` is a number and the checker can see that.
- The expression `(x) => e` is a lambda expression.
- The expression `x ?? y` is equal to `x`, unless `x` is `null` or `undefined`, in which case it is equal to `y`.
- The expression `c ? a : b` is the ordinary conditional expression on the level of values.

2 Structural typing

A type describes what fields a value has. Any value with those fields has that type, whether or not anybody said so. Nothing has to be declared and nothing has to be constructed.

For unions that has a cost, and an example makes it plain. Suppose a length is either in metres or in feet.

```
type Metres = { value: number };
type Feet   = { value: number };
type Length = Metres | Feet;
```

Those two are the same type. A value with one numeric field called `value` is a `Metres`, and it is equally a `Feet`, and the checker will let you pass either one where the other is expected. Given a `Length` there is no test that tells you which you have, because there is nothing to test.

In Haskell you would write `data Length = Metres Double | Feet Double`. The constructor is itself the answer. TypeScript has no constructors, so if you want to tell the two apart you must put something there to look at.

```
type Metres = { unit: "m"; value: number };
type Feet   = { unit: "ft"; value: number };
```

Now `x.unit === "m"` answers the question. But notice what `unit` is. It is an ordinary field holding an ordinary string. The language did not provide it. We added it, and we have to keep adding it, and everything that wants to tell metres from feet has to read it.

2.1 A tagged union writes down a finite choice

Once every case carries a tag, the checker can use it. Stay with lengths, and give the two cases different data so that there is something to get wrong.

```
type Length =
  | { unit: "m"; metres: number }
  | { unit: "ft"; feet: number };
```

3 Compiling down to JavaScript

3.1 A cast is a promise, not a check

The types are *erased*. The program that executes has no representation of them and no check derived from them. So the `as` operator is not a conversion. It is an assertion. The checker accepts it without verifying, and the runtime cannot verify it at all.

In the example below, `toFixed` is a method that rounds a number to a given number of decimal places. It exists on numbers and on nothing else, which is what makes it a useful test.

```
const raw = '{"id":"1","title":"Diaspora","author":"Greg Egan"}';
const b = JSON.parse(raw) as Book;    // a promise, not a check
console.log(b.id.toFixed(0));          // compiles: id is declared number
```

```
$ deno check t2.ts      -- passes. Nothing is wrong as far as the checker knows.
$ deno run    t2.ts
TypeError: b.id.toFixed is not a function
```

The checker was told `id` is a number and believed it. The `id` was a string all along. The error surfaced at the first place that actually used it. That may be a long way from the cast, and in a real program is likely to be somewhere else entirely.

3.2 `unknown` is what you want at a boundary

Two types describe a value you know nothing about, and they behave in opposite ways.

The type `any` switches the checker off. Every operation on an `any` is permitted.

```
const raw: any = JSON.parse('{"id":"1","title":"Diaspora","author":"Greg Egan"}');
console.log(raw.id.toFixed(0));
```

```
$ deno check anyex.ts      -- passes, no complaint
$ deno run    anyex.ts
TypeError: raw.id.toFixed is not a function
```

The `id` was the string `"1"` all along. Nothing objected, because nothing was looking.

The type `unknown` does the opposite. It permits nothing until you have established what you have.

```
const raw: unknown = JSON.parse('{"id":"1","title":"Diaspora","author":"Greg Egan"}');
console.log(raw.id);
```

```
$ deno check unknownex.ts
TS18046 [ERROR]: 'raw' is of type 'unknown'.
```

That refusal is the useful thing. The type `any` would have compiled and then failed later, in the manner of the previous subsection. The type `unknown` makes the boundary visible. It refuses

to let you cross it accidentally.

3.3 Validate, do not cast

So how does a value we know nothing about honestly become one whose type we know? By code that looks at the fields. The signature to aim for is `unknown` in, and the type *or nothing* out.

```
function asBook(u: unknown): Book | null {
  if (typeof u !== "object" || u === null) return null;
  const o = u as Record<string, unknown>;
  if (typeof o.id !== "number") return null;
  if (typeof o.title !== "string") return null;
  if (typeof o.author !== "string") return null;
  return { id: o.id, title: o.title, author: o.author }; // rebuilt, not asserted
}
```

Try it on three documents.

```
asBook({ id: 13, title: "Diaspora", author: "Greg Egan" })  
// { id: 13, title: "Diaspora", author: "Greg Egan" }  
  
asBook({ id: "13", title: "Diaspora", author: "Greg Egan" })  
// null -- the id is a string  
  
asBook({ id: 13, author: "Greg Egan" })  
// null -- there is no title
```